Testing

UNZA Hostel Maintenance Request and Tracking System

University of Zambia — Final Year Project

Testing

How the system is tested, what each kind of test proves, how to run them, and how they run automatically on every push.

Totals: 158 automated tests — 110 unit + 48 integration — plus a CI pipeline and browser-based verification of the front end.

---

1. Strategy: three levels

Level	What it tests	Speed	Where
Unit	One business rule in isolation (a service function or pure helper) with repositories mocked	milliseconds	`server/src/**/*.test.js`
Integration	The **real Express app over HTTP**: routes + middleware + services working together, with only the database and external services faked	~5 s for all	`server/tests/integration/`
Browser / manual	The React UI in a real browser (phone and desktop widths), offline behaviour, dark mode, live updates; full student → admin → technician walk-through (UAT)	seconds–minutes	Playwright scripts, manual demo

The architecture makes this possible: services contain no `req` / `res`, so they unit-test cleanly; the Express app is exported separately from the code that opens a port (`server/src/app.js` vs `server/server.js`), so it can be driven in-process without a network.

---

2. Running the tests

From the `server/` folder:

```
npm test                  # everything (unit + integration)
npm run test:unit         # business rules only
npm run test:integration  # HTTP-level flows only
```

No `.env`, database, network access or credentials are needed. That is deliberate: it is what allows the same commands to run in CI.

---

3. What the integration tests prove

Each request goes through the real `authenticate` middleware, `requireRole` guards, route validation, service business rules and the standard error formatter. Only these are replaced with fakes:

- **Supabase Auth**: a token like `student-token` resolves to a fake student, so authentication and role logic run for real.
- **Every repository** (the database layer) is mocked, so we can assert *what the service asked the database to do*.
- Push notifications and the AI client (side effects).

Area	Scenarios covered (48 tests)
Platform	Health endpoint is public · missing token → 401 · invalid token → 401 with the standard error shape · live-update stream requires auth
Role-based access	8 cases proving each role is refused (403) on endpoints it must not use, e.g. student → all-requests / assign / analytics, technician → submit a request, administrator → student-only endpoints
Request submission	Creates the request as `open` **and** logs the initial `status_history` row · always attributed to the *authenticated* student even if the body claims another id · validation (missing description, missing hostel, bad category, unknown priority → 400, nothing saved) · database foreign-key violation → clear 400 · photo uploaded first and URL stored · non-image "photo" rejected
Assignment	Creates the assignment, moves to `assigned`, logs history · refuses non-open requests · refuses a non-technician account · 404 for unknown request · 400 for malformed technician id
Lifecycle	Assigned technician can start work · a different technician gets 403 and nothing changes · skipping a step is rejected with an explanatory 400 · `open` / `assigned` cannot be set directly · student cannot change status · administrator can advance any request
Proof of work	Optional by default · photo and note are saved **before** the status changes · invalid photo is rejected and the request stays in progress
Read access	Owner sees their request with timeline and SLA · other student → 403 · only the assigned technician and admins can read it · 404 / 400 handling · a student's list is scoped to their own id
Upvotes and community feed	Back someone else's request · cannot upvote own or resolved requests · repeated upvote is harmless · removal works · staff cannot upvote · feed hides your own requests and **never leaks other students' names or ids**
AI triage	Suggests a category and forces hazards to critical (safety floor) · too-short description → 400 · rate limit → 429 after 10 a minute
Smart assignment	Technicians returned best-first with workload and one recommendation

The tests catch real bugs (mutation check)

Passing tests only matter if they *fail when the code is wrong*. We verified this by deliberately breaking two rules and re-running:

Deliberate bug	Result
Remove the administrator-only guard from `GET /api/requests`	2 access-control tests failed
Allow lifecycle steps to be skipped	the "skipping a step" test failed

The bugs were reverted and all tests pass again.

---

4. What the unit tests cover (110)

Lifecycle transitions and role restrictions · proof-of-work ordering and the optional/mandatory switch · SLA states and boundaries · overdue detection and alert batching · technician ranking · upvote rules and feed ranking · AI triage safety floor and rate limit · push delivery and dead-subscription cleanup · live-event routing · weekly-email statistics, schedule and HTML escaping · analytics maths (resolution time, workload, trends) · the authentication and role middleware.

---

5. Continuous integration (GitHub Actions)

`.github/workflows/ci.yml` runs automatically on every push to `main` and on every pull request:

```
push / pull request
   ├── Server job : npm ci → unit tests → integration tests
   └── Client job : npm ci → lint → production build
```

- A **red ✗** on a commit means something broke *before* it can reach the live site.
- A newer push to the same branch cancels the older run (saves time).
- It uses Node 22 and caches `node_modules` for speed.
- It needs **no secrets**, because the tests never touch a real database.
- The pipeline was rehearsed locally from a clean checkout with no `.env`: unit + integration tests, lint and build all passed (105 + 48 at the time; 110 + 48 now).

Optional next step: in Netlify/Render (or GitHub branch protection) require the CI check to pass before deploying or merging.

---

6. Adding a test

A new business rule → write a unit test next to the service (`something.service.test.js`), mocking repositories with `vi.mock(...)` .

A new endpoint or flow → add a case to `server/tests/integration/api.integration.test.js` :

```
const res = await request(app)
  .post('/api/requests/5/assign')
  .set({ Authorization: 'Bearer admin-token' })   // pick an identity
  .send({ technicianId: TECH.user_id });

expect(res.status).toBe(200);
expect(requestsRepository.updateRequestStatus).toHaveBeenCalledWith(5, 'assigned');
```

Identities available: `student-token`, `student2-token`, `admin-token`, `tech-token`, `tech2-token`. Set repository return values with `repository.fn.mockResolvedValue(...)`.

---

7. Honest limits

- The integration tests fake the database, so they prove the **application logic and wiring**, not Postgres itself: SQL, Row Level Security policies and migrations are verified by running them in Supabase and by manual UAT. A further step would be a disposable test database (e.g. a Supabase branch or Docker Postgres) for a small "real database" suite.
- **Front-end automation is not in CI.** The browser checks (Playwright) were run during development but are not yet a committed test suite.
- Push delivery, email delivery and the live Claude call need real credentials and devices, so they are verified manually.